

EXPERIMENT 4 REPORT

R. VIKAS RAJ
EE19B108, Batch-1

Objective:

The objective of this experiment is to implement and demonstrate the use of external interrupts INT0 and INT1 in the Atmel AVR ATmega microcontroller. The goal is to blink an LED 10 times, with a 1-second ON and 1-second OFF period, upon triggering either of the interrupts using push buttons.

Software and Hardware:

- **Microcontroller:** Atmel ATmega8
 - **Programmer:** USBASP
 - **Breadboard Setup:** Push buttons connected to INT0 (PD2) and INT1 (PD3), LED connected to PB0
 - **Software:** Microchip Studio (Atmel Studio), AVR Burn-o-mat
-

Procedure:

1. Setting Up the Interrupts:

○ INT0 (PD2):

- Configured for falling-edge detection using the **ISC01** bit in the MCUCR register.
- Enabled by writing to the GICR register (**GICR = 0x40**).

○ INT1 (PD3):

- Configured similarly for falling-edge detection by setting the **ISC11** bit in the MCUCR register.
- Enabled using the GICR register (**GICR = 0x80**).

2. Stack and Port Initialization:

- The stack pointer was initialized using the `LDI` and `OUT` instructions to set the `SPL` (0x70) and `SPH` (0x00) registers.
- `PB0` was configured as an output by setting the `DDRB` register (`DDRB = 0x01`), while all of `PORTD` was set as input by clearing `DDRD` (`DDRD = 0x00`).

3. Interrupt Service Routine (ISR):

- **INT0 ISR:**
 - When triggered, the ISR saves the state of the registers and makes the LED blink 10 times, with each cycle consisting of a 1-second ON and 1-second OFF period.
 - The delays are achieved through nested loops.
 - The ISR ends by restoring the registers and returning using the `RETI` instruction.
- **INT1 ISR:**
 - Similarly, the `INT1` interrupt causes the LED to blink for approximately 10 times using a delay loop inside the ISR.

4. Main Program:

- After configuring both interrupts and enabling global interrupts (`SEI`), the main program enters an infinite loop (`rjmp ind_loop`), waiting for external signals to trigger `INT0` or `INT1`.

Assembly Code:

INT0 Code:

```
.org 0x0000
    rjmp reset                ; Reset vector - jump to reset label
                                when the microcontroller starts

.org 0x0002
    rjmp int0_ISR            ; INT0 interrupt vector - jump to
                                ISR when INT0 interrupt occurs

.org 0x0100                    ; Start the program from address 0x0100
```

reset:

LDI R16, 0x70	; Load immediate value 0x70 into R16
OUT SPL, R16	; Set Stack Pointer Low byte to 0x70
LDI R16, 0x00	; Load immediate value 0x00 into R16
OUT SPH, R16	; Set Stack Pointer High byte to 0x00 (initialize stack pointer)
LDI R16, 0x01	; Load immediate value 0x01 into R16
OUT DDRB, R16	; Set PB0 as output (DDRB controls the data direction for Port B)
LDI R16, 0x00	; Load immediate value 0x00 into R16
OUT DDRD, R16	; Set all pins of PORTD as input (DDRD controls the data direction for Port D)
LDI R16, 0x02	; Load 0x02 into R16 (sets ISC01 bit for falling edge detection on INT0)
OUT MCUCR, R16	; Write the value to MCUCR (Control Register for Interrupts)
LDI R16, 0x40	; Load 0x40 into R16 (enables INT0 interrupt)
OUT GICR, R16	; Write the value to GICR (General Interrupt Control Register)
LDI R16, 0x00	; Load 0x00 into R16 (initialize PORTB to 0, turning off the LED)
OUT PORTB, R16	; Write value to PORTB (to control Port B output)
SEI	; Enable global interrupts

```

ind_loop:
    rjmp ind_loop        ; Infinite loop, wait for interrupts

int0_ISR:                ; ISR for INT0 interrupt
    PUSH R16             ; Save register R16 on the stack
    IN R16, SREG          ; Save the status register to R16
    LDI R16, 10           ; Load the blink count (10 times) into R16
    MOV R0, R16           ; Move the blink count into R0

blink_loop:
                                ; Turn ON LED

    LDI R16, 0x01         ; Load 0x01 into R16 (turn on LED connected
                                to PB0)

    OUT PORTB, R16        ; Write to PORTB to turn the LED on

                                ; 1-second ON delay

    LDI R16, 100          ; Outer loop counter for delay (100 iterations)
delay_on:
    LDI R17, 250          ; Inner loop counter for delay (250 iterations)
delay_on_inner:
    DEC R17               ; Decrement R17
    BRNE delay_on_inner   ; Repeat inner loop until R17 becomes 0
    DEC R16               ; Decrement outer loop counter
    BRNE delay_on         ; Repeat outer loop until R16 becomes 0

                                ; Turn OFF LED

    LDI R16, 0x00         ; Load 0x00 into R16 (turn off LED)
    OUT PORTB, R16        ; Write to PORTB to turn the LED off

                                ; 1-second OFF delay

    LDI R16, 100          ; Outer loop counter for delay (100 iterations)
delay_off:
    LDI R17, 250          ; Inner loop counter for delay (250 iterations)

```

```

delay_off_inner:
    DEC R17                ; Decrement R17
    BRNE delay_off_inner   ; Repeat inner loop until R17 becomes 0
    DEC R16                ; Decrement outer loop counter
    BRNE delay_off         ; Repeat outer loop until R16 becomes 0

    DEC R0                 ; Decrement the blink count in R0
    BRNE blink_loop        ; If blink count is not zero, repeat blinking

    POP R16                ; Restore register R16 from the stack
    OUT SREG, R16          ; Restore status register
    RETI                   ; Return from the interrupt

```

INT1 Code:

```

.org 0x0000                ; Set the reset vector at address 0x0000
rjmp reset                 ; Jump to the reset routine at program start

.org 0x0004                ; Set the vector for INT1 (external interrupt 1)
                             ; at address 0x0004
rjmp int1_ISR              ; Jump to the INT1 interrupt service routine
                             ; (ISR) when INT1 occurs

.org 0x0100                ; Start the main program at address 0x0100
Reset:
                             ; Initialize the stack pointer to 0x70 (Stack
                             ; Pointer Low)

    LDI R16,0x70           ; Load immediate value 0x70 into register R16
    OUT SPL,R16            ; Output R16 to SPL (Stack Pointer Low)

                             ; Initialize the Stack Pointer High to 0 (Stack
                             ; Pointer High)

```

```

LDI R16,0x00      ; Load immediate value 0x00 into register R16
OUT SPH,R16       ; Output R16 to SPH (Stack Pointer High)

                    ; Configure PORTB pin 0 as an output (to
                    ; control an LED)

LDI R16,0x01      ; Load immediate value 0x01 into register R16
                    ; (set PB0 as output)
OUT DDRB,R16      ; Output R16 to DDRB (Data Direction Register B)

                    ; Configure MCUCR register for falling edge
                    ; detection on INT1
LDI R16,(1<<ISC11) ; Load value with ISC11 bit set (falling edge
                    ; detection)
OUT MCUCR,R16     ; Output R16 to MCUCR (MCU Control Register)

                    ; Enable INT1 interrupt in the GICR register
LDI R16,(1<<INT1)  ; Load value with INT1 bit set to enable INT1
                    ; interrupt
OUT GICR,R16      ; Output R16 to GICR (General Interrupt Control
                    ; Register)

                    ; Initialize PORTB to 0 (turn off the LED
                    ; initially)
LDI R16,0x00      ; Load immediate value 0x00 into R16
OUT PORTB,R16     ; Output R16 to PORTB (set all bits low, turn
                    ; off LED)

SEI               ; Enable global interrupts (Set Interrupt Enable
                    ; flag)

ind_loop:
    rjmp ind_loop ; Infinite loop to keep the program running,
                    ; waiting for an interrupt

```

```

; Interrupt Service Routine (ISR) for INT1

int1_ISR:

; Save the Status Register (SREG) to R16 and
push it onto the stack

IN R16,SREG      ; Copy the Status Register into R16
PUSH R16         ; Push R16 onto the stack to preserve the
                ; original SREG

; Blink LED connected to PB0 for approximately 1
second

LDI R16,0x0A     ; Load 10 into R16 (10 blink cycles)
MOV R0,R16       ; Move the blink count from R16 into R0 for
                ; later use

c1:              ; Start the blinking loop

; Turn on the LED by setting PB0 high

LDI R16,0x01     ; Load 0x01 into R16 (set PB0 high)
OUT PORTB,R16    ; Output R16 to PORTB (turn on the LED)

; Delay loop for ~1 second (outer loop)

LDI R16,0xFF     ; Load 0xFF into R16 (outer delay counter)

a1:              ; Load 0xFF into R17 (inner delay counter)

LDI R17,0xFF

a2:              ; Decrement the inner delay counter (R17)
DEC R17          ; If R17 is not zero, branch back to a2
BRNE a2          ; (continue inner loop)

DEC R16          ; Decrement the outer delay counter (R16)
BRNE a1          ; If R16 is not zero, branch back to a1
                ; (continue outer loop)

```

	; Turn off the LED by setting PB0 low
LDI R16,0x00	; Load 0x00 into R16 (set PB0 low)
OUT PORTB,R16	; Output R16 to PORTB (turn off the LED)
	; Restore the Status Register (SREG) from the stack
POP R16	; Pop the saved value from the stack into R16
OUT SREG,R16	; Output R16 to SREG (restore original SREG)
RETI	; Return from the interrupt and restore normal execution

Results:

- **INT0 Trigger:** Upon pressing the push button connected to PD2 (INT0), the LED blinked 10 times with a 1-second ON and 1-second OFF cycle.
- **INT1 Trigger:** Similarly, pressing the button connected to PD3 (INT1) triggered the LED to blink 10 times with the same cycle.

Both INT0 and INT1 were successfully implemented, and the LED blinking behavior demonstrated the correct functioning of the interrupt routines.

Conclusion:

The experiment was successful in demonstrating the configuration and functionality of external interrupts (INT0 and INT1) in the Atmel AVR microcontroller. The assembly code effectively responded to external push-button triggers, controlled by the interrupt service routines, resulting in a blinking LED. This showcases the efficiency of interrupt-driven programming in microcontroller applications.

Snapshots:

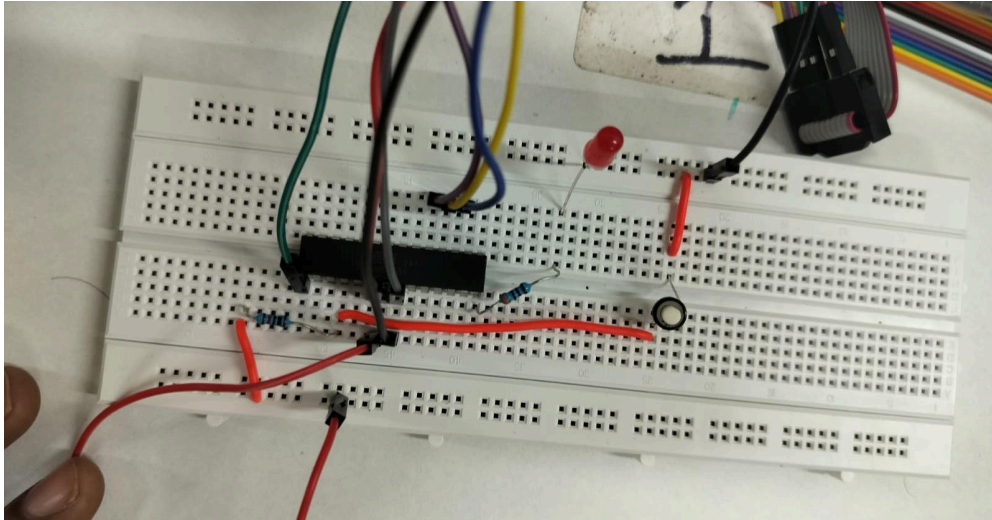


Fig-2: before turning push-button on(int0)

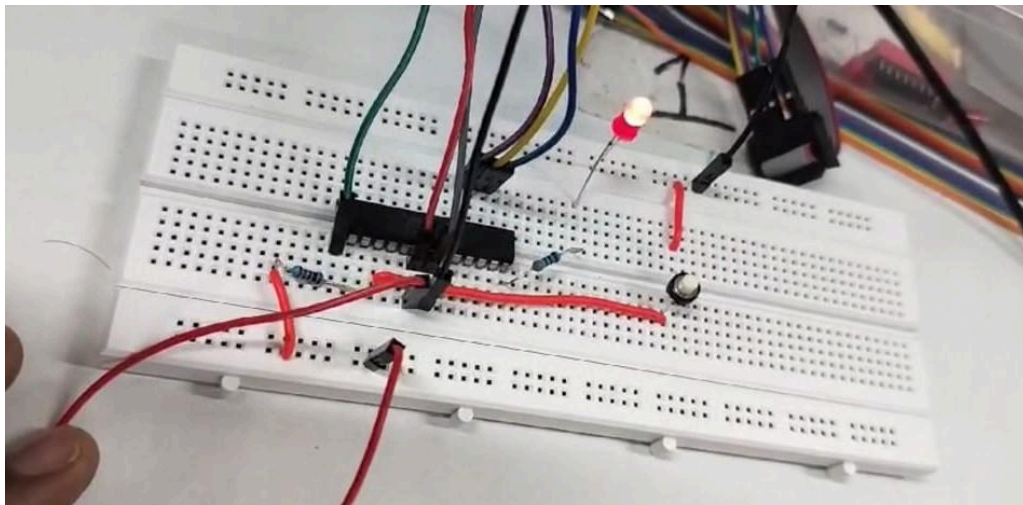


Fig-2: After turning push-button on(int0)

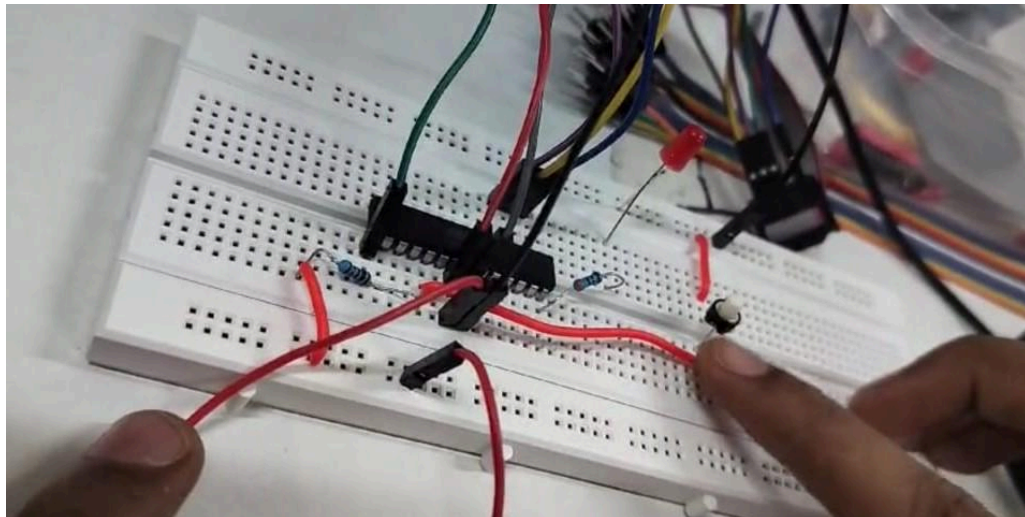


Fig-3:Before turning the switch on(int1)

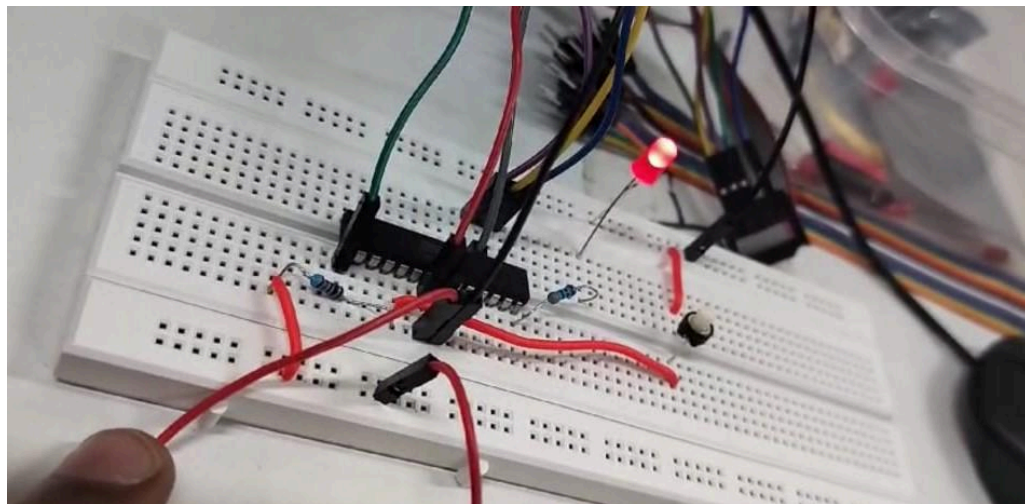


Fig-4: After turning the switch on(int1)